



DEVELOPER TECHNICAL NOTE

Saathi

Milk Traceability Pilot — Pour-to-Pack Provenance, Sachiv Logging &
Farmer Services • Uttar Pradesh / PCDF-Parag

CLASSIFICATION
Strictly Confidential

VERSION
v0.1

DATE
7 July 2026

PREPARED BY
PYAAS

CONFIDENTIAL

CONFIDENTIALITY NOTICE

This document and its contents are the confidential and proprietary property of PYAAS and are disclosed strictly on a confidential basis solely for evaluation and internal use in connection with the Saathi platform. It may contain privileged, commercially sensitive, or pre-decisional information. No part of this document may be copied, reproduced, distributed, disclosed, published, or transmitted to any third party, in whole or in part, in any form or by any means, without the prior written consent of PYAAS. If you have received this document in error, please notify the sender immediately and destroy all copies. Unauthorised use, retention, or disclosure may be unlawful and may result in liability.

Saathi Platform — Developer Technical Note

Milk Traceability Pilot: Pour-to-Pack Provenance, Sachiv Logging & Farmer Services

Prepared by Pyaas for PCDF (Parag) · Nand Baba Dugdh Mission **Status:** Engineering specification — Pilot build · **Classification:** Confidential

How to read this note

This is the **build-facing** companion to the *Saathi Architecture Framework*. The Framework sets the platform-wide posture (three-tier cooperative model, national DPI integration, identity/RBAC, compliance). This note goes one level deeper on the **single capability the pilot must prove**: that a carton of Parag milk can be traced, honestly and verifiably, back through the plant, the collection van, and the village society to the farmers who poured it — and that the same rails carry fat/SNF capture, rate calculation and farmer payment.

It is written so an engineer can start building: it fixes the domain model, the event/provenance ledger, the API surface, the state machines, the offline-sync contract, the security model, and the acceptance tests. Where a design choice is a genuine trade-off, that is stated rather than hidden.

One principle governs the whole document, and it is stated up front because everything else follows from it:

Milk is pooled. Traceability of a pooled product is set-valued, not point-valued. A pack cannot honestly claim "this came from one farmer's cow." It can honestly claim "the milk in this batch was collected in this window from *these* farmers across *these* societies, and passed *these* quality checks." The system is engineered to make that set-valued claim precise, tamper-evident and fast to resolve — and to refuse to imply more than the physics allows. §12 treats this in full.

Table of contents

1. Pilot objective and the traceability claim
 2. Actors, apps and roles
 3. The golden path (end-to-end)
 4. Domain model
 5. The provenance ledger (append-only, hash-linked)
 6. Feature specifications - 6.1 Farmer profile and services (cattle, helpline, schemes, videos) - 6.2 Sachiv logging (fat / SNF / quantity / rate) - 6.3 Rate and digital-invoice engine - 6.4 Collection, batch creation and transport tracking - 6.5 Plant intake, the quality gate and the production lot - 6.6 QR generation and printing - 6.7 Consumer resolution (what a scan reveals)
 7. State machines
 8. API surface
 9. Offline-first and synchronisation
 0. Data integrity, security and compliance
 1. Technology stack and deployment
 2. Honest limitations and threat model
 3. Pilot acceptance criteria (definition of done)
 4. Build phasing - Appendix A — Identifier and code formats - Appendix B — QR payload and resolution - Appendix C — Rate-chart model - Appendix D — Glossary
-

1. Pilot objective and the traceability claim

The pilot runs in two districts (Lucknow and Gonda) for the defined window. It is judged on one question: **can we stand behind what the QR says?**

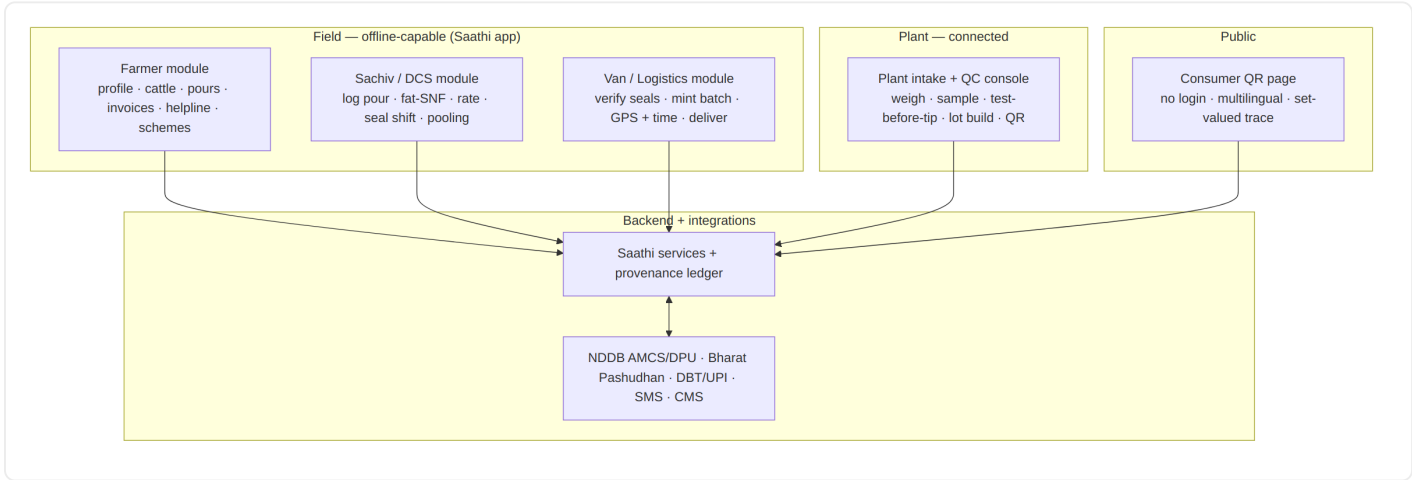
Concretely, the pilot must demonstrate four things working together on real milk:

- Capture at source is trustworthy enough to price and pay on.** Every farmer pour at a society is logged with quantity, fat and SNF, at a recorded time, by a known Sachiv, on a known device — with an explicit *assurance level* attached so the data's provenance is itself auditable.
- The physical chain of custody is recorded as it happens.** Pooling at the society, pickup by the van (which mints a **batch code**), the van's route and timing, arrival at the plant, and the quality gate — each is an event, ordered, signed and immutable.
- The pack carries a claim the system can defend.** A unique QR on the pack resolves to the exact production lot, and from there to the societies and farmers whose milk it contains, and to the quality result — no more, no less.
- The rails also pay the farmer.** The same pour data drives a fat/SNF-based rate and a per-cycle digital invoice, settling to the farmer's bank via DBT.

What the pilot is **not** trying to prove: that a single sachet is attributable to a single animal. That is neither physically true of pooled, silo-blended, standardised milk nor claimed. See §12.

2. Actors, apps and roles

The platform is one system with role-scoped surfaces, not five apps. A person authenticates once (phone + OTP, layered onto the cooperative's member/identity records) and receives the modules their role grants. A Sachiv is often also a farmer; a supervisor may cover several societies. Identity and RBAC are specified in the Framework; this note uses the roles below.



Role	Primary surface	Core capability	Trust level
Farmer / Producer	Farmer module	View own pours, invoices, cattle, services	Self-scoped read
Sachiv (DCS Secretary)	Sachiv module	Log pours, apply rate, seal the shift	Write within own society
Van driver / Logistics	Van module	Verify society seals, mint batch, carry GPS/time	Write within own trip
Plant operator / QA	Intake + QC console	Weigh, sample, gate quality, build lot, issue QR	Write within own plant
Supervisor / Union	Web console	Oversight, corrections approval, audit	Cross-society read + approve
Consumer	Public QR page	Resolve a pack to its provenance set	Anonymous read
Administrator	Web console	Master data, rate charts, content, keys	Governed by change control

3. The golden path (end-to-end)

The narrative below is the reference flow every component must honour. Times are indicative of a morning (AM) shift.

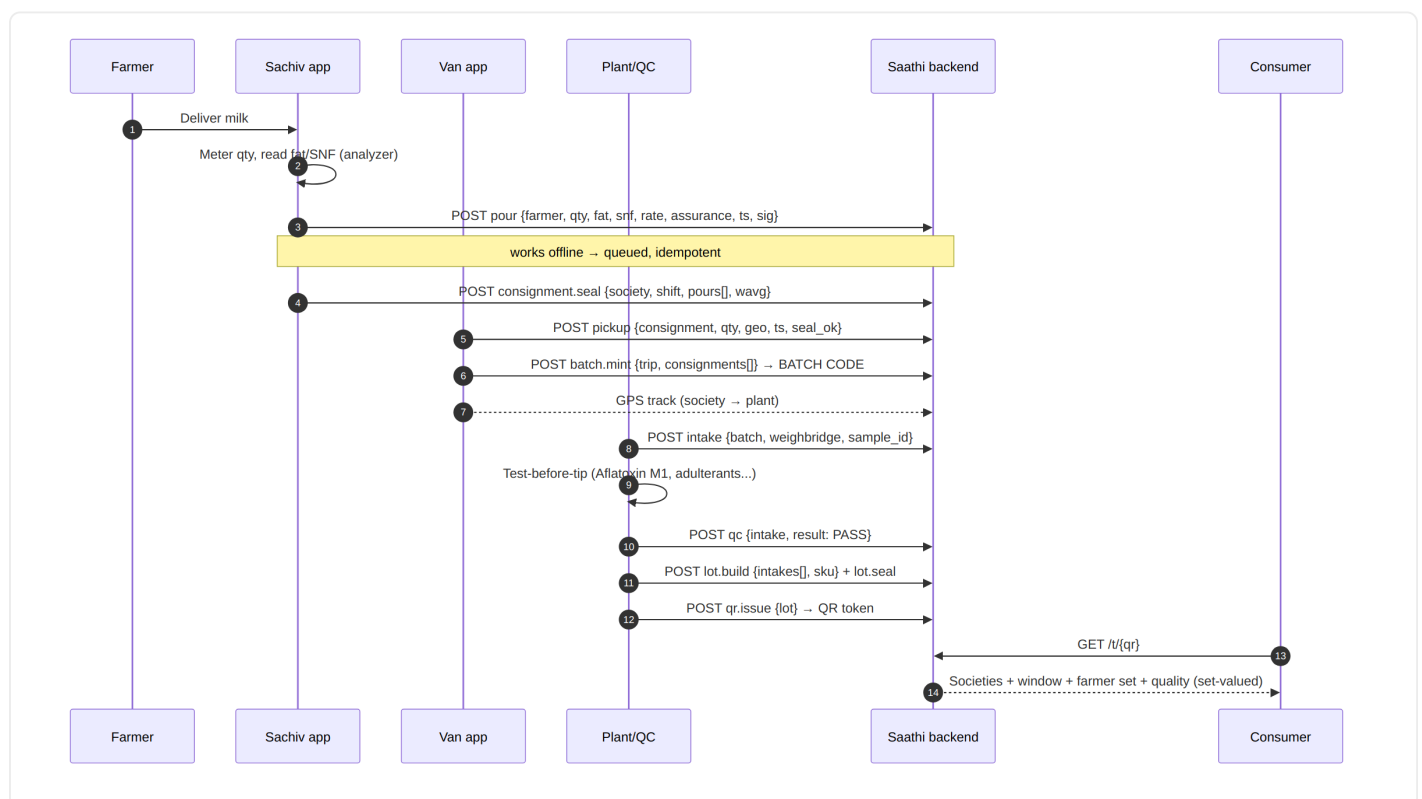
A farmer arrives at the society with milk. The Sachiv identifies the member, weighs or meters the milk, and the analyzer reads fat and SNF. The app computes the rate from the current chart and records a **pour** — an immutable event tied to farmer, society, shift, device and time. The farmer sees the entry (and later the invoice) in their module; a printed/So SMS slip is optional.

At the end of the shift the Sachiv **seals** the society's pooled milk into a **consignment**: the app freezes the pour set, computes quantity-weighted average fat/SNF, and produces a seal code for the physical cans. No pour can be silently altered after sealing (§5, §7).

The collection van runs its route. At each society the driver verifies the seal, and the app records the pickup — quantity, time, GPS. When the van closes its pickups for the trip, the system **mints a batch code** binding all the consignments it carries. The van's movement is tracked as a GPS/time track from society to plant.

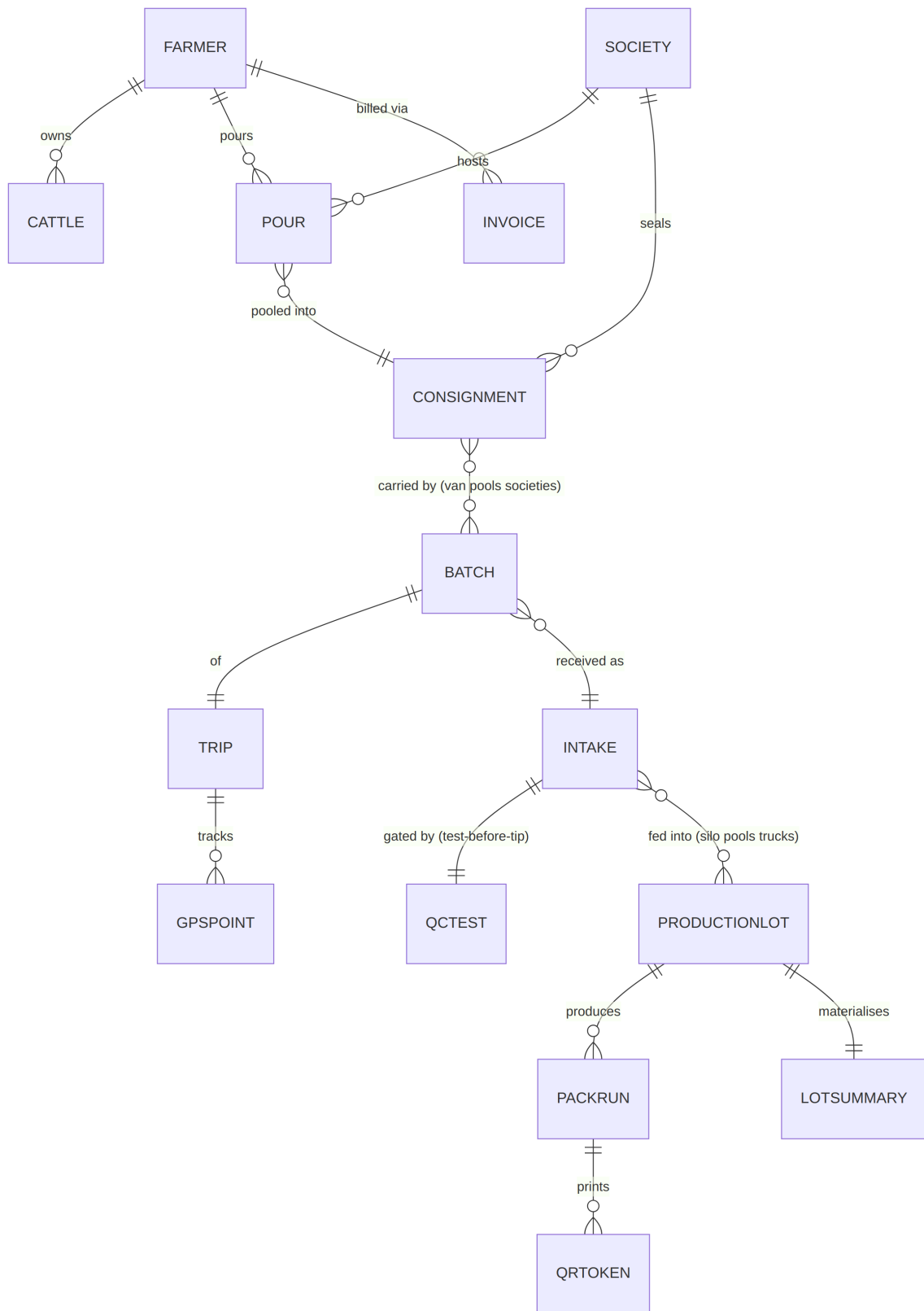
At the plant, each incoming batch is weighed and **sampled before it is tipped into any common silo** — the quality gate. Aflatoxin M1, adulteration, added water, antibiotic residue and microbial checks decide PASS / HOLD / REJECT. Only PASS milk enters the silo. The plant then draws from silos to run a **production lot** of a specific SKU; the lot references the intakes that fed it. Packaging prints a **QR** bound to that lot.

A consumer scans the QR. The service resolves the lot to the set of intakes → batches → consignments → pours, and returns a consumer-appropriate, privacy-respecting view: the societies (on a map), the collection window, the count and roster of contributing farmers (identities shown only with consent), and the quality result.



4. Domain model

Entities are **events** (immutable facts, the spine of traceability) and **records** (mutable master/reference data). Traceability is preserved by never mutating an event and by materialising the aggregation edges as first-class links.



The full relationship graph is above; the field-level schema for each entity follows. (event) marks append-only, hash-chained custody events (\$5); the rest are records.

FARMER	farmer_id, member_code, society_id, name, phone_masked, bank_ref_masked, cattle_count, kyc_status, public_consent
CATTLE	cattle_id, farmer_id, species, breed?, pashu_aadhaar?(null), tag_status[untagged tagged verified], added_at
POUR (event)	pour_id, farmer_id, society_id, shift_date, shift, qty_l,

	fat_pct, snf_pct, rate_per_l, amount, rate_chart_version,
	assurance[A B C], source, device_id, captured_at,
	prev_hash, event_hash, device_sig
CONSIGNMENT	consignment_id, society_id, shift_date, shift, total_qty_l,
(event)	wavg_fat, wavg_snf, seal_code, sealed_at, event_hash
BATCH (event)	batch_code, trip_id, picked_qty_l, minted_at, minted_by, event_hash
TRIP	trip_id, vehicle_id, driver_id, route_code, started_at,
	plant_arrival_at, plant_id
GPSPPOINT	trip_id, lat, lng, at, source
INTAKE (event)	intake_id, batch_code, plant_id, weighbridge_l, sample_id, received_at
QCTEST (event)	qc_id, intake_id, fat, snf, aflatoxin_ml_ppb, added_water_pct,
	adulterants, antibiotic, result[PASS HOLD REJECT], tested_at, tested_by
PRODUCTIONLOT	lot_id, plant_id, sku, process_type, qty_produced_l, sealed_at
(event seal)	
QRTOKEN	token, lot_id, pack_run_id, serial, issued_at, status
LOTSUMMARY	lot_id, societies[], farmer_roster[], collection_window,
(materialised)	farmers_total, quality_result
INVOICE	invoice_id, farmer_id, society_id, cycle_id, period, lines[],
	gross, deductions[], net_payable, pdf_ref, pdf_hash,
	payment_status, payment_ref, issued_at

Notes that matter for correctness:

- **POUR is the atomic traceability unit and is append-only.** A mistake is fixed by appending a correction/reversal event that references the original, never by editing (§5, §7). `assurance` records *how* fat/SNF was obtained (instrument-linked / OCR-with-photo / manual); `source` records the device path.
- **Two aggregation fan-ins exist and both are recorded.** Pours fan into a consignment (one society, one shift). Consignments fan into a batch (one van trip, several societies). Batches fan into intakes at the plant, and **intakes fan into a production lot (silo blending across many trucks)**. The lot is therefore the union of a very large farmer set — this is the physical reason traceability is set-valued (§12).
- **LOTSUMMARY is a materialised roll-up** computed at lot seal: the distinct societies, the farmer roster with per-farmer contribution, the collection window, and the quality result — so a consumer scan is O(1) and does not walk the graph live.

5. The provenance ledger (append-only, hash-linked)

Traceability is only as good as the guarantee that a recorded fact was not altered after the fact. Saathi stores every custody event in an **append-only ledger** with per-event hash-chaining and device signatures. This gives tamper-evidence without the operational weight of a public blockchain; an external anchor is an optional phase-2 hardening (§14).

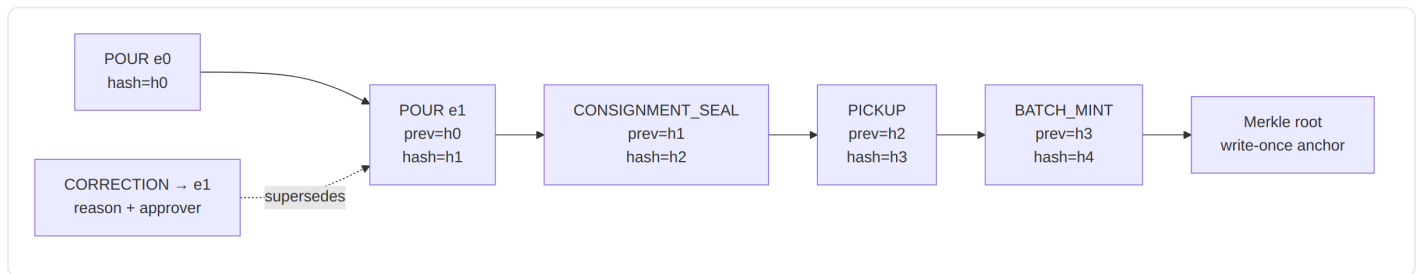
Event envelope. Every field/plant event is written as:

```
{
  "event_id": "uuid",           // server canonical id
  "client_event_id": "uuid",    // client-generated, for idempotency
  "type": "POUR | CONSIGNMENT_SEAL | PICKUP | BATCH_MINT | INTAKE | QC | LOT_SEAL | QR_ISSUE | CORRECTION",
  "actor_id": "uuid",          // authenticated principal
  "device_id": "string",
  "occurred_at": "RFC3339",     // device clock at capture
  "received_at": "RFC3339",     // server clock at ingest
  "payload": { /* type-specific, see §4 */ },
  "prev_hash": "hex(sha256)",   // previous event in this chain
  "event_hash": "hex(sha256)",  // = sha256(prev_hash || canonical_json(payload_core))
}
```

```
"device_sig": "ed25519(...)" // signature over event_hash by the device key
}
```

Chaining. Each society, each trip and each plant line maintains its own chain (so an offline device can extend its local chain without contacting the server). On sync, the server verifies `event_hash` and `device_sig`, links the event into the authoritative per-chain sequence, and periodically writes a Merkle root over all chains to an internal, write-once anchor table (and, in phase 2, an external timestamping authority). Any later attempt to alter an event breaks its hash and every hash after it, and fails signature verification.

Corrections. A `CORRECTION` event references the `event_id` it supersedes and carries the corrected payload plus a reason and an approver. Same-shift, pre-seal corrections may be made by the Sachiv within a grace window; post-seal corrections require Union approval and are always visible in the audit trail. Nothing is ever deleted; the "current" value of a pour is the head of its correction chain.



6. Feature specifications

6.1 Farmer profile and services

The farmer module is the trust surface for the producer. It is deliberately light — it must run on an entry-level Android phone, in Hindi, often on a weak connection — and it does four jobs: show who you are on the system, show what you delivered and are owed, connect you to help, and connect you to schemes and learning.

Profile and cattle. The profile shows the member code, society, village, KYC status, "member since", and the **cattle count**. Cattle is a simple counter for the pilot, backed by an optional per-animal registry so it can later bind to the national animal identity (12-digit Pashu Aadhaar under Bharat Pashudhan / NDLM). Binding animals to the national ear-tag ID is what makes a future herd-health / collar / per-animal yield capability possible; the pilot keeps that path **present but dormant behind a feature flag**, storing `pashu_aadhaar` on `CATTLE` when available and never inventing it.

```
CATTLE { cattle_id, farmer_id, species, breed?, pashu_aadhaar? (nullable),
  tag_status: [untagged|tagged|verified], added_at }
```

Editing cattle count is a low-risk, self-service write; changes are logged (not append-only ledgered — cattle is master data, not a custody event).

Helpline. A single "Get help" screen with click-to-call and click-to-ticket, populated from region config so numbers can change without an app release:

- Society Sachiv (the farmer's own society contact)
- Milk Union / district help desk
- **1962** Mobile Veterinary Unit (animal emergency)
- Pyaas / Saathi support line
- In-app ticket (category, free text, optional photo) → routed to Union desk with SLA tracking

Numbers and labels come from the CMS (below), scoped by society/district, cached offline, and dialled via the OS dialer (`tel:` intent) so no calling infrastructure is needed for the pilot.

Schemes and educational videos. These are **CMS-driven content**, never hardcoded links, so the Union/administrator can curate per region and per language and update instantly. Two collections render as simple, cached lists with thumbnails:

- **Schemes** — deep links to the Nand Baba Dugdh Mission portal, the state Animal Husbandry & Dairying department, NDDDB producer resources, DBT/subsidy and insurance schemes, etc.
- **Learning** — a curated video playlist (clean-milk production, mastitis prevention, cattle nutrition and fodder, artificial insemination and breeding, calf care), deep-linked to the hosting platform; thumbnails and titles are cached for offline browsing, playback opens the external app/browser.

```
CONTENT { content_id, type: [scheme|video|article|helpline],
          title_i18n, description_i18n, url, thumbnail_url,
          languages[], region_scope: [state|district|society|all],
          category, order, published, valid_from, valid_to }
```

Content sync is a versioned delta pull (`GET /content?since=<version>`), so the app fetches only what changed; the last-good set is always available offline.

6.2 Sachiv logging (fat / SNF / quantity / rate)

This is where trustworthy data enters the system, so it gets the most care.

For each pour the Sachiv (a) identifies the farmer (search by code/name, or scan the member card; recent farmers cached for the shift), (b) captures **quantity** (from the weigh scale/DPU, or manual with a reason), (c) captures **fat and SNF**, and (d) lets the app compute the **rate and amount** from the active chart. The pour is written to the local ledger immediately and syncs when possible.

The critical design decision is the **fat/SNF source and its assurance level**, because milk payment fraud usually lives here:

Assurance	How fat/SNF is obtained	Field reality	Recorded as
A — Instrument-linked	Milk analyzer streams the reading to the app over RS-232 / BLE (NDDDB AMCS / DPU integration)	Automated societies	<code>assurance=A</code> , analyzer device id captured
B — OCR-with-evidence	Sachiv photographs the analyzer display; the app OCRs the value and stores the photo as evidence	Semi-automated societies	<code>assurance=B</code> , image ref stored
C — Manual	Sachiv keys the value	Manual societies / fallback	<code>assurance=C</code> , flagged for higher audit

Level A is preferred and is the target end-state. Level B exists because a large share of societies still read a standalone analyzer; capturing the *photo of the screen* alongside the OCR'd number gives an audit trail — while being explicit (see §12) that a photo of a screen is spoofable and therefore lower-assurance, not proof. Level C is a last resort and drives supervisor sampling. Every pour carries its level, and every downstream aggregate (consignment, batch, lot) carries the **weakest** assurance in its set, so the pilot can measure and improve capture quality rather than paper over it.

Validation at capture: quantity and fat/SNF are range-checked against configurable bounds and against the farmer's rolling history; out-of-band values require confirmation and are flagged. Duplicate-pour protection uses `client_event_id` + (farmer, shift) so a double-tap or a retry after a dropped connection cannot create two pours.

6.3 Rate and digital-invoice engine

Rate. The rate is a pure function of fat, SNF and the **versioned rate chart** the Union publishes. Two chart models are supported (the Union picks one; both are data, not code):

- **Two-axis matrix** — `rate_per_l = chart[fat_band][snf_band]` .

- **Kg-Fat / Kg-SNF formula** — $\text{amount} = \text{kg_fat} \times \text{rate_fat} + \text{kg_snf} \times \text{rate_snf}$, where kg values derive from quantity, density and fat/SNF %.

Charts are versioned with `valid_from` / `valid_to`; every pour stores the `rate_chart_version` it was priced on, so an invoice is always reproducible and a mid-cycle chart change never silently rewrites history. See Appendix C.

Invoice. Dairy co-ops settle on a cycle (commonly 10-day). At cycle close the engine produces, per farmer, a **digital invoice**: all pours in the cycle (date, shift, qty, fat, SNF, rate, amount), gross payable, configurable **deductions** (cattle feed, AI/veterinary services, advances/loans), net payable, and the payment reference. The invoice is:

- viewable in the farmer module (line-by-line, downloadable),
- issued as a **signed PDF** (server-side, immutable, hash recorded),
- summarised over SMS for farmers who don't open the app,
- settled via **DBT** to the farmer's bank (integration per the Framework's payments section — bank/PFMS/UPI), with the payment result written back to the invoice.

```
INVOICE { invoice_id, farmer_id, society_id, cycle_id, period,
          lines[] (pour_id, qty, fat, snf, rate, amount),
          gross, deductions[], net_payable,
          pdf_ref, pdf_hash, payment_status, payment_ref, issued_at }
```

Because invoices are built only from immutable, hash-chained pours, a farmer dispute can be resolved by replaying the exact pours and the exact chart version — the invoice is *defensible*, which is the point.

6.4 Collection, batch creation and transport tracking

When the van reaches a society, the driver's module **verifies the seal** the Sachiv applied (scan/enter `seal_code`; mismatch is flagged and blocks a clean pickup), then records the **pickup**: quantity taken, timestamp, and GPS location. When the driver closes pickups for the trip, the system **mints the batch code** — the pilot's pivotal object — binding every consignment the van carries into one traceable unit (Appendix A for the format). The batch is signed and immutable.

Two aspects the build must get right:

- **The van pools too.** One batch typically contains several societies' consignments (a milk route). So the batch → consignments → pours edges are the record that lets a later scan name *all* contributing societies and farmers for that leg. Pickup order and per-society quantities are retained.
- **Transport is tracked as time + location.** The trip carries a GPS track (points at a sensible cadence, buffered offline and uploaded on connectivity) plus discrete stop events (each society pickup) and the plant-arrival event. This is what substantiates "collected at ~06:40, reached plant at ~08:15" on the consumer view, and what lets operations spot cold-chain or timing anomalies. GPS gaps are recorded as gaps, not interpolated silently (§12).

6.5 Plant intake, the quality gate and the production lot

The plant is where food safety and traceability intersect, and the ordering is deliberate and non-negotiable:

Test before tip. A representative sample is drawn and the quality gate evaluated **before** a batch is emptied into any common silo. Only PASS milk is tipped. This is what protects the entire pooled lot from a single bad intake — and it is essential for heat-stable hazards.

At intake: the batch is weighed (weighbridge/flow), a `sample_id` is drawn, and the intake event is recorded against the batch. QC then evaluates, at minimum: **Aflatoxin M1** (FSSAI limit 0.5 µg/kg; heat-stable, so it *must* be caught here — pasteurisation will not remove it), added water (freezing-point depression), adulterants (neutralisers, detergents, urea, starch, etc.), antibiotic/veterinary residue, and microbial indicators appropriate to the product. The result gates the milk:

- **PASS** → tip into silo, intake becomes eligible to feed a lot.
- **HOLD** → segregate, re-test or escalate; not eligible until resolved.
- **REJECT** → divert/destroy per SOP; recorded against the batch (and surfaced to the Union and, where appropriate, the originating society for corrective action).

A **production lot** is then built by drawing from silos to run a specific SKU; the lot records which **intakes** fed it. This is the second, largest fan-in — a silo blends many trucks, so a lot's farmer set is the union across all its intakes. At **lot seal**, the **LOTSUMMARY** is materialised (societies, farmer roster with contributions, collection window, lot-level quality result), which is what the consumer endpoint reads.

6.6 QR generation and printing

Each packaging run against a sealed lot issues QR tokens. The QR encodes **only a short, signed URL** — never farmer data — so the consumer view can evolve, be revoked, and stay small and printable:

```
https://trace.parag.in/t/<token>
<token> = base62(lot_ref) · short-HMAC (Appendix B)
```

Two granularities are supported:

- **Lot-level QR** (pilot default) — all packs in the run share one token that resolves to the lot's provenance set. Simplest to print and sufficient to prove traceability.
- **Serialised per-pack QR** (phase-2 anti-counterfeit) — each pack gets a unique serial under the same lot, enabling scan-count analytics and clone detection.

The HMAC makes tokens **unguessable and self-verifying** (a random or altered token fails the check before any lookup), which blocks enumeration and casual forgery. Tokens can be revoked (e.g., a recalled lot) and will then resolve to a clear recall notice rather than provenance.

6.7 Consumer resolution (what a scan reveals)

A scan hits a **public, no-login, multilingual, cache-friendly** endpoint that returns the materialised summary — the honest, set-valued provenance:

- the **product / SKU**, the **pack/lot** and the **collection window** (dates/times);
- the **societies** that contributed, shown on a **map at society granularity** (never farmer homes);
- the **number of contributing farmers**, and a **roster** — identities (name/village/photo) shown **only for farmers who have consented** (DPDP); non-consenting farmers are counted but not named;
- the **quality result** (a "passed FSSAI checks" badge with the tested parameters at a lay level);
- links to schemes/verification and a short, honest explanation that milk is pooled.

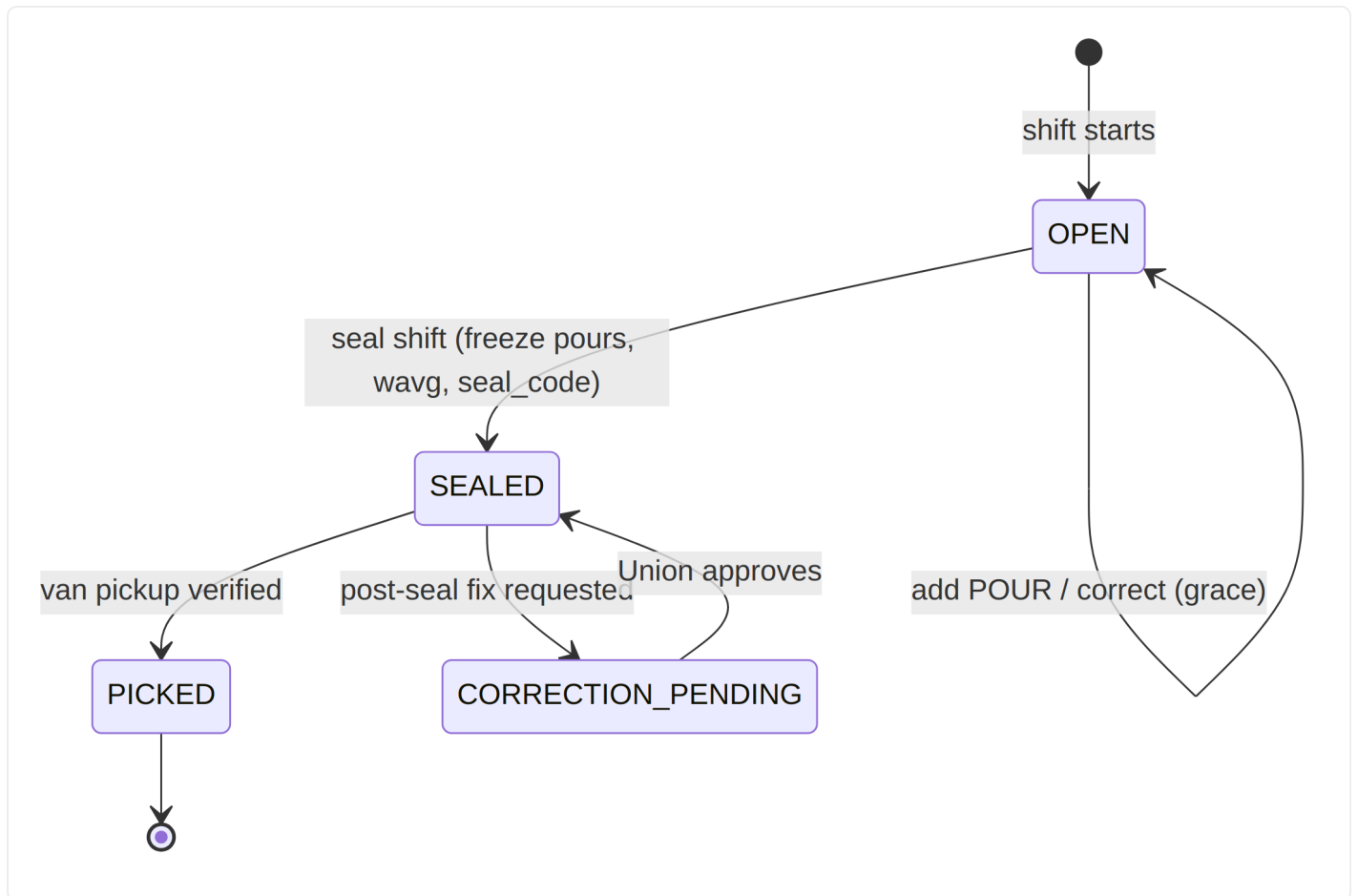
```
// GET /t/{token} → 200
{
  "product": { "sku": "PARAG-TONED-500ML", "name": "Parag Toned Milk 500 ml" },
  "lot": { "id": "LOT-LK0-1-20260705-014", "packed_on": "2026-07-05" },
  "collection_window": { "from": "2026-07-04T05:30+05:30", "to": "2026-07-04T18:10+05:30" },
  "societies": [
    { "code": "GON-DCS-0421", "name": "...", "village": "...", "geo": [27.13, 81.96], "farmers": 92 },
    { "code": "GON-DCS-0388", "name": "...", "village": "...", "geo": [27.20, 81.88], "farmers": 74 }
  ],
  "farmers_total": 812,
  "farmers_public": [
    { "name": "...", "village": "...", "cattle": 3, "member_since": 2019, "photo": "..." }
  ],
  "quality": { "result": "PASS", "aflatoxin_m1": "within limit",
    "adulteration": "not detected", "tested_on": "2026-07-05T08:40+05:30" },
```

```
"disclosure": "This batch blends milk pooled from the societies and farmers listed above."  
}
```

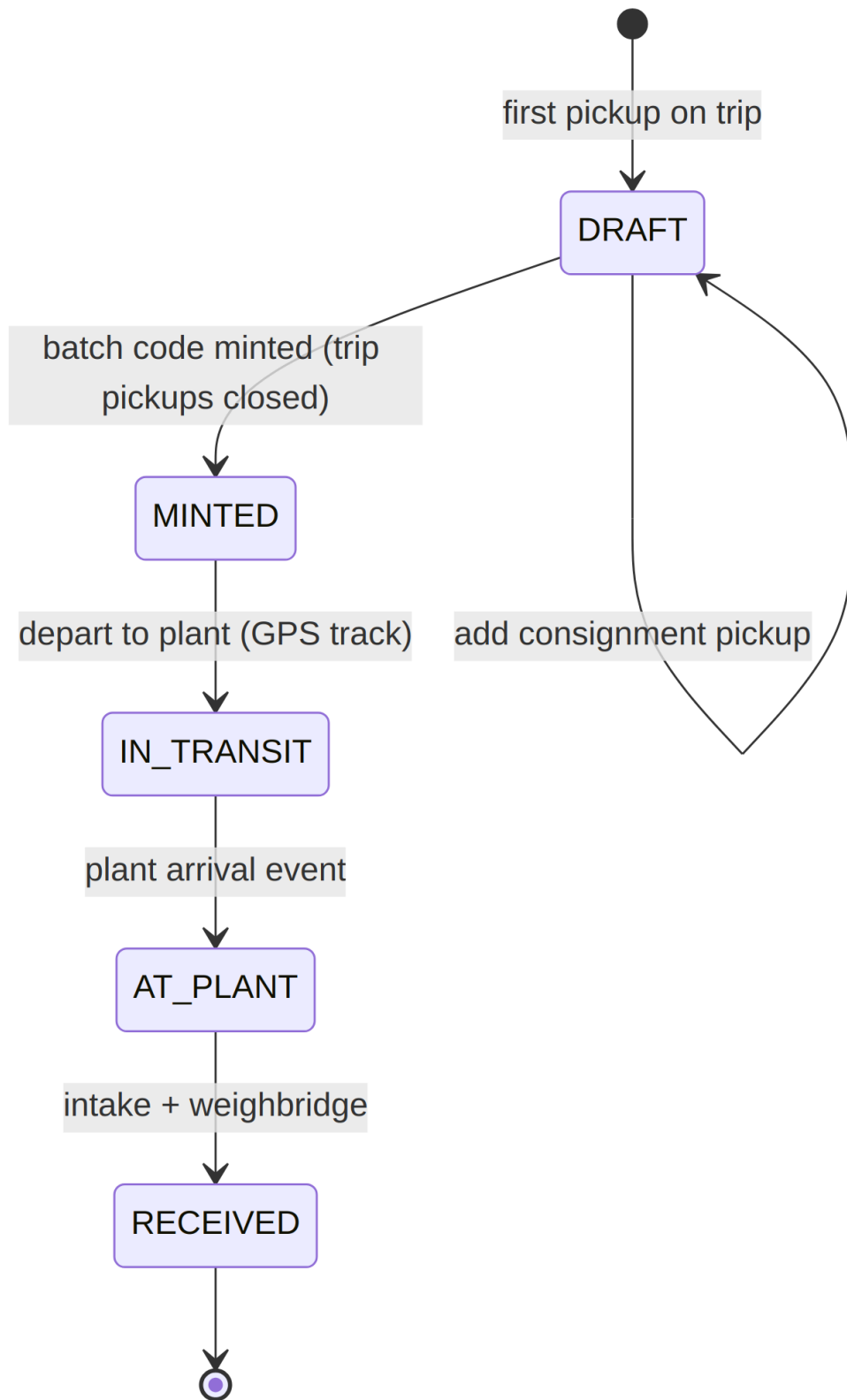
The copy is written to make the true claim compelling — "this batch was poured by 812 farmers across 9 societies in Gonda; passed quality checks on 5 July; meet some of them" — rather than a false single-source claim.

7. State machines

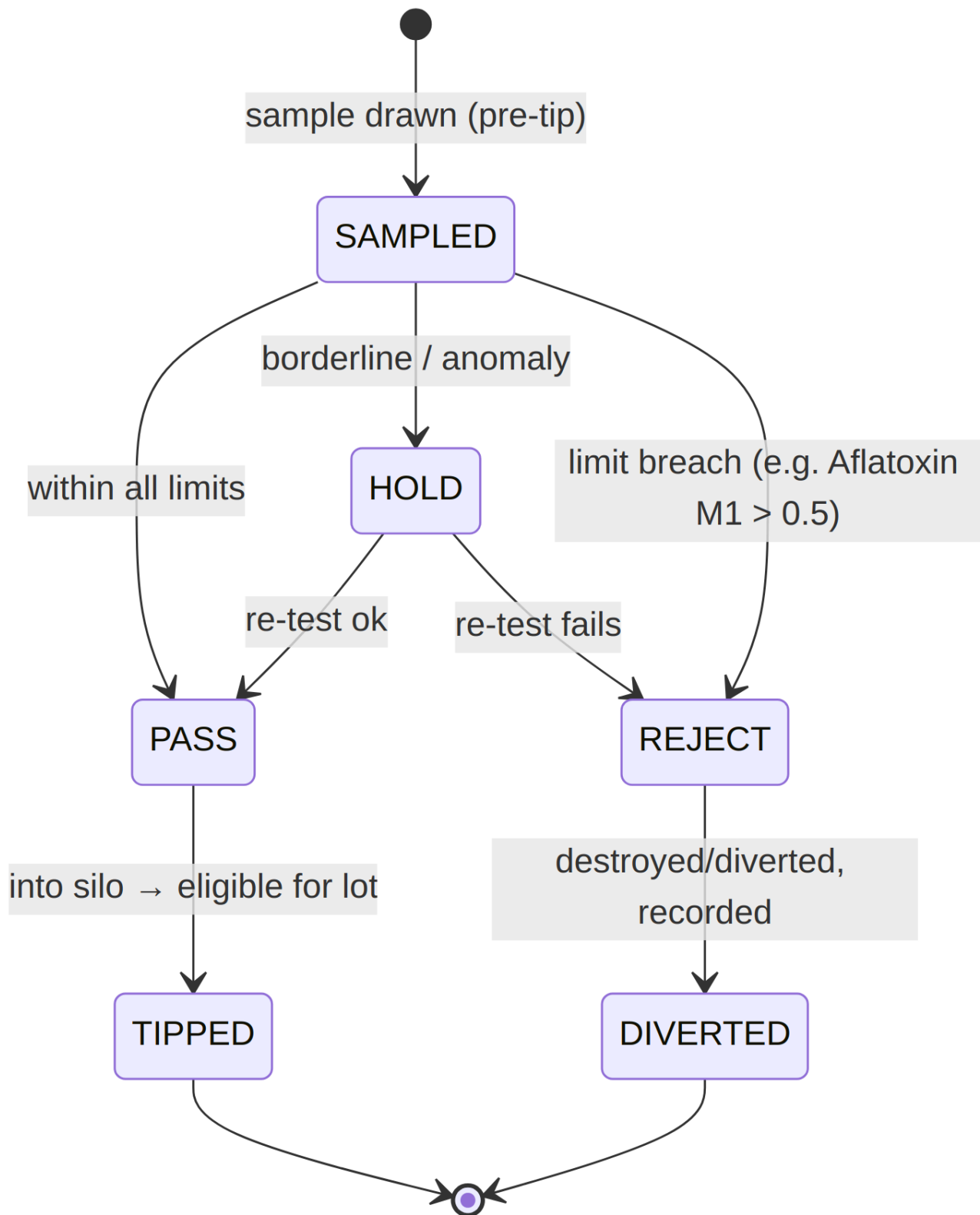
Consignment (the society shift). A shift accumulates pours, then seals; sealing is the point after which the pour set is frozen and corrections require approval.



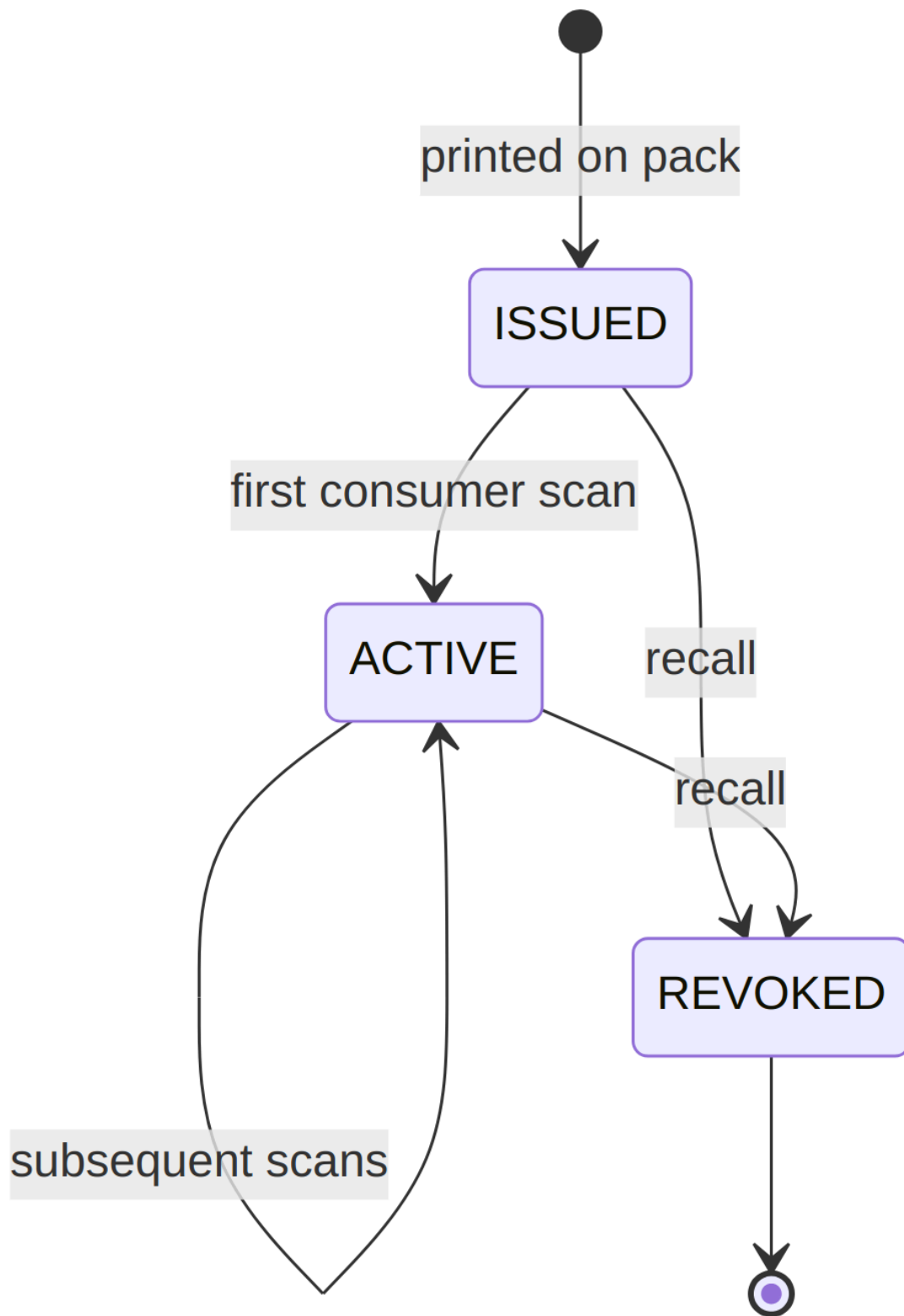
Batch and transport.



Quality gate (per intake) — decides whether milk enters the pool.



QR token.



8. API surface

REST/JSON over TLS. All writes are **idempotent** (carry `client_event_id`; replay returns the original result). Field writes are accepted optimistically and reconciled on sync (§9). Representative endpoints:

Method	Path	Purpose	Who
POST	/pours	Record a pour (qty, fat, snf, rate, assurance, sig)	Sachiv
POST	/pours/{id}/corrections	Append a correction to a pour	Sachiv/Union
POST	/consignments/{id}/seal	Seal the shift; freeze pours; wavg; seal_code	Sachiv
POST	/pickups	Van pickup of a consignment (qty, geo, ts, seal_ok)	Van
POST	/batches/mint	Mint batch code over a trip's consignments	Van
POST	/trips/{id}/track	Append buffered GPS points	Van
POST	/intakes	Plant intake of a batch (weighbridge, sample_id)	Plant
POST	/qc	QC result for an intake (PASS/HOLD/REJECT + params)	Plant/QA
POST	/lots/build · /lots/{id}/seal	Build/seal a production lot over intakes	Plant
POST	/qr/issue	Issue QR token(s) for a pack run	Plant
GET	/t/{token}	Public provenance resolution (materialised)	Consumer
GET	/farmers/{id}/pours?cycle=	Farmer's pours for a cycle	Farmer
GET	/invoices/{id} · /farmers/{id}/invoices	Invoice detail / list	Farmer
POST	/invoices/run	Close a cycle, generate invoices	Union
GET	/content?since= · /config/helpline?scope=	CMS content / helpline numbers (delta)	Farmer
POST	/sync/batch	Bulk push of queued offline events	Sachiv/Van
GET	/reference?since=	Delta pull of rate charts, roster, config	Field apps

Example — record a pour (idempotent, offline-friendly):

```
POST /pours
{
  "client_event_id": "1b9d...c2",
  "farmer_id": "...", "society_id": "...",
  "shift_date": "2026-07-04", "shift": "AM",
  "qty_l": 8.4, "fat_pct": 4.1, "snf_pct": 8.6,
  "rate_chart_version": "GON-2026-06",
  "assurance": "A", "source": "analyzer:DPU-2231",
  "captured_at": "2026-07-04T06:12:05+05:30",
  "prev_hash": "...", "event_hash": "...", "device_sig": "..."
}
→ 201 { "pour_id": "...", "rate_per_l": 41.2, "amount": 346.08, "assurance": "A" }
```

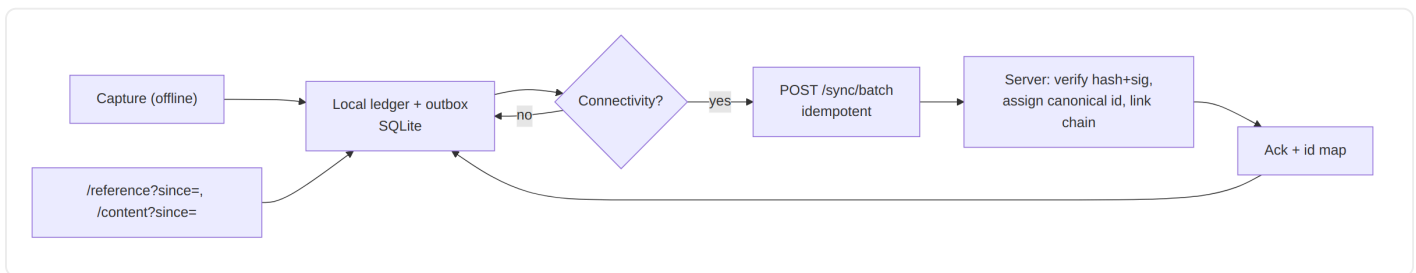
9. Offline-first and synchronisation

Rural connectivity is the norm, not the exception, so the Sachiv and Van modules are **offline-first by contract**, not as a fallback. NDDB's own field experience — intermittent power and network at societies — is the reason capture, sealing and pickup must all complete with no server present.

Design:

- **Local source of truth on device** (SQLite/Drift). Every capture writes to the local hash-chained ledger and an **outbox**. The UI never blocks on the network.

- **Outbox / at-least-once sync.** Queued events push on connectivity via `/sync/batch`; idempotency keys make replays safe. The server assigns canonical `event_id`s and links events into authoritative chains; the device maps its temp ids to canonical ids on ack.
- **Append-only ⇒ largely conflict-free.** Because pours are immutable facts (not mutable rows), concurrent capture across devices does not conflict. The few mutable interactions (a post-seal correction, a cattle-count edit) are resolved by explicit rules (approval for the former; last-writer-wins with audit for the latter), never by silent overwrite of custody data.
- **Reference data is versioned and delta-pulled.** Rate charts, farmer roster, helpline numbers and CMS content sync as deltas (`?since=`), with the last-good set retained for offline use so a device is never left without a chart to price on.
- **Clocks.** Capture stores the device clock (`occurred_at`) and the server stores ingest time (`received_at`); the van's GPS supplies satellite time for transport. Ordering within a chain is by sequence, not wall-clock, so clock skew cannot reorder custody.



10. Data integrity, security and compliance

The platform handles farmer identity, bank references and location, and it makes public claims about food — so integrity and privacy are first-order requirements. This section states the pilot's obligations; the Framework holds the full control set.

Integrity / tamper-evidence. Hash-chained, device-signed, append-only events with periodic Merkle anchoring (§5). Corrections are additive and audited. Invoices are immutable signed PDFs with recorded hashes. This makes both payment and provenance *defensible under dispute*.

Identity and data minimisation (DPDP Act 2023 + Rules 2025). Aadhaar (used for member eKYC/DBT per the Framework) is **never displayed or stored in the clear** — it is tokenised/masked and held in a protected vault, with only the reference on the profile. Phone and bank references are masked in the app. The public QR view exposes farmer identity **only with explicit consent** (`public_consent`); otherwise farmers are counted, not named. No farmer home location, phone, bank or Aadhaar ever appears on the public endpoint — society geo only.

Access control. Party- and role-scoped RBAC (a Sachiv writes only within their society; a driver only within their trip; a plant operator only within their plant); every privileged action is audit-logged. Public endpoints are read-only, rate-limited and cache-fronted.

Transport / at rest. TLS in transit; encryption at rest for PII and evidence images; signed URLs and short-lived tokens for media; secrets and device keys in managed key storage.

Food-safety thresholds as data. QC limits (Aflatoxin M1 ≤ 0.5 µg/kg, added-water, adulteration, residues, microbial) are configuration, versioned like rate charts, so a regulatory change updates the gate without a code change — and every QC result stores the limit-set version it was judged against.

Logging / incident. Security-relevant logs are retained per CERT-In direction (including the required retention window), with an incident-reporting path. Data residency in an India region per MeitY guidance (Framework §hosting).

11. Technology stack and deployment

The recommendation below optimises for the pilot's real constraints — entry-level Android, weak networks, a small team that must ship and operate — and for **integrating with, not rebuilding, the cooperative's existing NDDB/AMCS estate**. Final choices should defer to PCDF's current stack where one exists.

- **Field apps (Farmer / Sachiv / Van):** a single **Flutter** app with role-scoped modules — one codebase, strong offline story (SQLite/Drift), good performance on low-end Android, Hindi/English from day one, and hardware access for BLE/serial analyzers and the camera (OCR + evidence). Native Android (Kotlin) is an acceptable alternative if deeper serial/peripheral work demands it.
- **Plant intake + QC console:** a responsive **web** app (operators are at fixed stations, connected) with weighbridge/analyzer integration and label/QR printing.
- **Consumer QR page:** a lightweight, statically-cacheable **web** page behind a **CDN**, reading only the materialised **LOTSUMMARY**.
- **Backend:** containerised services in a **boring, robust** runtime (Java/Spring Boot or Node.js/TypeScript). **PostgreSQL + PostGIS** as the system of record (append-only event tables + master data + geo), **Redis** for cache/queues, an event-ingest queue (**Kafka/RabbitMQ**) to absorb sync bursts at scale, and **object storage** for evidence images, invoice PDFs and QR assets. Read replicas / materialised summaries serve the public endpoint.
- **Integrations:** NDDB **AMCS/DPU** (fat/SNF/qty at automated societies), **Bharat Pashudhan / Pashu Aadhaar** (cattle identity, dormant path), **DBT / bank / UPI** (invoice settlement), **SMS gateway** (slips, invoice summaries, OTP), and the **CMS** (schemes, learning, helpline).
- **Hosting:** an India-region, **MeitY-empanelled** cloud for data residency; keys in managed KMS; IaC + CI/CD; observability (metrics, logs, traces) from day one.

12. Honest limitations and threat model

This is the most important section for anyone who has to *defend* the QR, and it is written plainly.

1. Traceability is set-valued because milk is pooled — twice. Milk is pooled at the society (many farmers → one consignment) and again in the plant silo (many trucks → one lot). A sealed pack therefore corresponds to a *set* of farmers and societies, not one. The system records the set precisely and the consumer view states it. It cannot, and does not claim to, attribute a specific pack to a specific farmer or animal. Any UI or marketing that implies single-source attribution for a pooled SKU would be false; the copy is written to make the *true* set-valued claim compelling instead.

2. Standardised product ≠ pour-level fat/SNF. Toned/standardised milk is adjusted at the plant, so the fat/SNF a farmer was paid on is **not** a description of the carton's fat/SNF. Pour-level fat/SNF is a payment fact; product composition is a separate, plant-level fact. The consumer view does not present pour fat/SNF as the pack's composition.

3. Capture assurance varies and is labelled, not hidden. Where fat/SNF is read from a standalone analyzer via a *photo of the screen* (assurance B) or keyed manually (assurance C), the number is only as trustworthy as the person and process. Storing the photo helps audit, but **a photo of a screen is spoofable** — it is evidence, not proof. Aggregates inherit the weakest assurance in their set, so the pilot can measure capture quality and drive societies toward instrument-linked (A) capture rather than pretend the problem away.

4. The digital record binds to physical milk only as tightly as seals and weights allow.

Batch/consignment integrity rests on seal verification and weighbridge reconciliation. Physical substitution or tampering between recorded events is a real-world risk mitigated by seal checks, quantity reconciliation at each hop, and anomaly flags — not eliminated by software. Large unexplained qty deltas between pour-sum, pickup, and weighbridge are surfaced for investigation.

5. GPS has gaps. Transport tracking depends on device GPS and connectivity; gaps happen (dead zones, dead batteries). Gaps are recorded as gaps and never interpolated into a false continuous track. Timing

claims on the consumer view come from recorded stop/arrival events, which are more robust than a dense track.

6. QR anti-forgery is proportionate, not absolute. HMAC-signed tokens stop enumeration and casual forgery, and revocation handles recalls. A determined counterfeiter can still photocopy a legitimate label onto fake product; **serialised per-pack tokens with scan-count/clone detection** (phase 2) raise that bar but do not make it infinite. This is stated so no one over-claims the QR as tamper-proof.

7. Consent limits the public roster by design. Non-consenting farmers are counted but not shown. This is a deliberate DPDP posture, and it means the public roster is a consented subset, not the full set — which the copy makes clear ("meet *some* of the farmers").

Where each risk is mitigated: seals + weighbridge reconciliation + anomaly flags (4), assurance levels + supervisor sampling + drive to instrument capture (3), test-before-tip + versioned limits (food safety), hash-chain + signatures + anchoring (record integrity), consent + masking + vault (privacy), HMAC + revocation + optional serialisation (QR).

13. Pilot acceptance criteria (definition of done)

The pilot is "done" when, on real milk in the two districts, all of the following hold:

1. **Source capture.** $\geq$ target % of pours captured at assurance A (instrument-linked); every pour carries farmer, society, shift, time, device, rate-chart version and assurance; duplicate-pour rate ≈ 0 under flaky-network testing.
2. **Seal integrity.** 100% of shifts seal with a quantity-weighted average and a seal code; post-seal edits occur only via approved, audited corrections.
3. **Batch minting.** Every van trip mints a batch code binding its consignments; seal-mismatch and qty-delta anomalies are detected and flagged; a batch resolves to its exact consignment and pour sets.
4. **Transport.** Trips carry GPS + stop/arrival events; timing on the consumer view matches recorded events; gaps are shown as gaps.
5. **Quality gate.** 100% of intakes are **sampled before tip**; Aflatoxin M1 and the configured panel gate PASS/HOLD/REJECT; a REJECT never enters a lot; limit-set version is recorded on every result.
6. **Lot → QR.** Every packaged lot issues a verifiable QR; a scan resolves in $< \sim 1s$ (materialised) to the correct societies, farmer count/roster, collection window and quality result — and to nothing beyond.
7. **Honesty.** The consumer view makes only the set-valued claim; no single-farmer attribution and no pour-fat-as-product-fat anywhere.
8. **Payment.** A cycle closes into per-farmer digital invoices reproducible from immutable pours and the recorded chart; invoices settle via DBT with results written back; a sampled dispute is resolved by replay.
9. **Offline.** A full shift (capture → seal) and a full pickup complete with no connectivity and reconcile correctly on reconnect, with no data loss and no duplicates.
10. **Privacy/security.** No Aadhaar/phone/bank/home-geo on any public surface; non-consenting farmers unnamed; RBAC and audit logging verified; residency and CERT-In logging in place.

14. Build phasing

- **Phase 0 — Foundations.** Identity/RBAC, master data (societies, farmers, roster, rate charts, QC limit sets), the event ledger + hash-chaining, sync contract, CMS. Field app shell with offline ledger.
- **Phase 1 — Capture & pay.** Sachiv logging (assurance A/B/C), rate engine, shift sealing, farmer module (profile, cattle, pours, helpline, schemes/videos), invoice engine + DBT + SMS.
- **Phase 2 — Chain of custody.** Van module (seal verify, pickup, **batch mint**, GPS/time), plant intake + weighbridge, **test-before-tip** QC gate, production-lot build/seal, **LOTSUMMARY** materialisation.
- **Phase 3 — The pack claim.** QR issue (lot-level), consumer resolution page (map, roster, quality, honest copy, multilingual), revocation/recall path.

- **Phase 4 — Hardening & scale.** Serialised per-pack QR + clone detection, external Merkle anchoring, analytics/oversight dashboards, and the dormant **Pashu Aadhaar / herd** path enabled where the national tagging is present.

Appendix A — Identifier and code formats

Object	Format	Example
Pour	UUID (opaque)	1b9d...c2
Consignment (society shift lot)	CON-<societyCode>-<YYYYMMDD>-<AM/PM>	CON-GON-DCS-0421-20260704-AM
Seal code (physical cans)	SEAL-<consignment>-<checksum>	SEAL-...-7F3A
Batch (van pickup)	BAT-<unionCode>-<routeCode>-<YYYYMMDD>-<shift>-<seq>·<hmac6>	BAT-LK0-R07-20260704-AM-014·9C2F1A
Intake	INT-<plantCode>-<YYYYMMDD>-<seq>	INT-LK001-20260705-233
Production lot	LOT-<plantCode>-<line>-<YYYYMMDD>-<seq>	LOT-LK001-1-20260705-014
QR token	base62(lot_ref)·short-HMAC (in a short URL)	.../t/9x4Kd2Qa7

Human-readable segments aid operations; the trailing HMAC on the batch code and QR token provides integrity/anti-forgery.

Appendix B — QR payload and resolution

The QR contains a short URL only: `https://trace.parag.in/t/<token>`, where `<token> = urlsafe(base62(lot_ref) || HMAC_k(lot_ref)[:6])`.

Resolution: the service verifies the HMAC (reject before any DB hit on failure) → looks up the sealed lot → reads the materialised `LOTSUMMARY` → applies consent/privacy filtering → returns the consumer payload (\$6.7). Revoked tokens short-circuit to a recall notice. All provenance data stays server-side so the view can change, be revoked, or be localised without reprinting anything.

Appendix C — Rate-chart model

```
RATE_CHART { version, union_scope, model: [matrix|kgfat_kgsnf],
              valid_from, valid_to,
              matrix?: { fat_band × snf_band → rate_per_l },
              params?: { rate_fat, rate_snf, density } }
```

- **Matrix:** `rate_per_l = matrix[band(fat)][band(snf)]`; `amount = qty_l × rate_per_l`.
- **Kg-Fat/Kg-SNF:** `kg_fat = qty_l × density × fat%/100`, `kg_snf = qty_l × density × snf%/100`, `amount = kg_fat × rate_fat + kg_snf × rate_snf`.

Every pour stores `rate_chart_version`, so pricing is reproducible and chart changes never rewrite history.

Appendix D — Glossary

Samiti / DCS — village Dugdh Utpadak Sahkari Samiti (dairy cooperative society), the collection point. **Sachiv** — the society Secretary who runs collection. **Pour** — one farmer's delivery at one shift (the atomic traceability unit). **Consignment** — a society's sealed, pooled milk for one shift. **Batch** — the van's minted unit binding the consignments it carries on a trip. **Intake** — a batch received and weighed at the plant. **Test-before-tip** — sampling and gating quality *before* milk enters a common silo. **Production lot** — a processed SKU run drawing from silos, referencing its intakes. **LOTSUMMARY** — the materialised, consumer-facing provenance roll-up. **Assurance level** — how fat/SNF was captured (A instrument-linked, B OCR-with-photo, C

manual). **Pashu Aadhaar** — the 12-digit national animal ID (Bharat Pashudhan / NDLM). **DBT** — Direct Benefit Transfer to the farmer's bank.

Confidential — prepared by Pyaas for PCDF (Parag) under the Nand Baba Dugdh Mission. This note is the engineering companion to the Saathi Architecture Framework; where the two overlap, the Framework governs platform-wide posture and this note governs the traceability build.